

Test-Time Training auto-supervisé pour la classification des images

Maksym Dolhov & Fallou Diouf

Encadrants: Pr Ayoub Karine (ayoub.karine@u-paris.fr)

Pr Camille Kurtz (camille.kurtz@u-paris.fr)

Pr Laurent Wendling (Laurent.Wendling@u-paris.fr)

Master 1 Vision et Machine Intelligente

Université Paris Cité

Paris, France

Résumé—Le décalage de distribution entre les données d’entraînement et les données de test constitue un problème majeur en vision par ordinateur. Les approches de *Test-Time Training* (TTT) visent à adapter un modèle directement au moment de l’inférence à l’aide d’objectifs auto-supervisés, sans accès aux annotations des données de test.

Dans ce travail, nous étudions l’intégration du TTT dans un pipeline de classification d’images fondé sur l’apprentissage auto-supervisé. Nous proposons un pipeline complet combinant un pré-entraînement SimCLR avec un encodeur ViT-Tiny, un fine-tuning supervisé sur CIFAR-10, puis une phase d’adaptation au test sur CIFAR-10-C à l’aide d’une tâche auxiliaire de prédiction de rotation.

Nos expériences montrent que le TTT en mode *per-batch* améliore systématiquement les performances sous dérive de distribution, avec des gains croissants lorsque la sévérité des corruptions augmente. À l’inverse, le TTT *per-image* dégrade les performances dans notre configuration basée sur ViT et LayerNorm. Les ablations montrent que des adaptations plus agressives amplifient simultanément les gains du mode batch et les dégradations du mode image.

Ces résultats suggèrent que l’efficacité du TTT dépend fortement du régime d’adaptation et de l’architecture de normalisation utilisée.

Index Terms—Self-Supervised Learning, Test-Time Training, Vision Transformer, Distribution Shift, Image Classification, Computer Vision

I. INTRODUCTION

Les modèles modernes de vision par ordinateur reposent généralement sur de grandes quantités de données annotées afin d’obtenir des performances élevées. Cependant, l’annotation manuelle des données demeure coûteuse et difficilement scalable. L’apprentissage auto-supervisé (*Self-Supervised Learning*, SSL) constitue une alternative prometteuse permettant d’apprendre des représentations générales à partir de données non annotées.

Dans un pipeline classique, un modèle est d’abord pré-entraîné de manière auto-supervisée sur un grand ensemble d’images, puis spécialisé via un fine-tuning supervisé pour une tâche de classification. Toutefois, les distributions des données d’entraînement et de test peuvent différer significativement. Ce phénomène, appelé *distribution shift*, entraîne souvent une dégradation importante des performances lors de l’inférence.

Le *Test-Time Training* (TTT) proposé par Sun et al. [1] vise à atténuer ce problème en adaptant le modèle directement pendant la phase de test à l’aide d’une tâche auto-supervisée auxiliaire. Contrairement aux approches classiques, le modèle continue donc à apprendre pendant l’inférence, sans accès aux labels des données de test.

Dans ce travail, nous étudions l’application du TTT dans un cadre combinant :

- un pré-entraînement auto-supervisé SimCLR ;
- un encodeur Vision Transformer (ViT-Tiny) ;
- une évaluation robuste sur CIFAR-10-C ;
- une adaptation au test basée sur une tâche de rotation.

Nos contributions principales sont les suivantes :

- 1) Nous proposons un pipeline complet SSL + TTT reproductible basé sur SimCLR et ViT-Tiny.
- 2) Nous comparons deux régimes d’adaptation au test : *per-batch* et *per-image*.
- 3) Nous montrons expérimentalement que le TTT *per-batch* améliore systématiquement les performances sous corruption, tandis que le TTT *per-image* dégrade fortement les performances dans notre configuration ViT/LayerNorm.
- 4) Nous analysons l’effet des hyperparamètres d’adaptation et montrons qu’une adaptation plus agressive amplifie simultanément les gains du mode batch et les dégradations du mode image.

II. TRAVAUX CONNEXES

A. Apprentissage auto-supervisé

L’apprentissage auto-supervisé vise à apprendre des représentations sans annotations explicites en construisant automatiquement des tâches prétextes à partir des données. Parmi les approches récentes les plus influentes figurent SimCLR [3], MoCo [4], BYOL [5] et DINO [6].

SimCLR repose sur un apprentissage contrastif où deux vues augmentées d’une même image doivent produire des représentations proches dans l’espace latent.

La perte NT-Xent utilisée par SimCLR est définie par :

FIGURE 1 – Vue d’ensemble du pipeline SSL + TTT proposé.

$$\mathcal{L}_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

où :

- z_i et z_j sont les projections de deux vues positives ;
- $\text{sim}(\cdot)$ désigne la similarité cosinus ;
- τ représente le paramètre de température.

B. Test-Time Training

Le Test-Time Training (TTT) introduit une phase d’adaptation directement pendant l’inférence [1]. L’idée principale consiste à exploiter une tâche auto-supervisée auxiliaire disponible au test afin d’adapter le modèle à la distribution courante.

Dans le travail original de Sun et al. [1], une tâche de prédiction de rotation est utilisée comme objectif auxiliaire. Le modèle est optimisé sur cette tâche avant de produire la prédiction finale de classification.

TTT++ [2] a ensuite étudié les conditions de succès et d’échec du TTT, mettant en évidence plusieurs problèmes de stabilité et de dérive du modèle.

Des travaux récents ont également étudié l’interaction entre TTT et les modèles auto-supervisés modernes [7], ainsi que des méthodes d’alignement de distributions telles que Act-MAD [8].

III. MÉTHODOLOGIE

A. Vue d’ensemble du pipeline

Notre pipeline est composé de trois étapes principales :

- 1) Pré-entraînement auto-supervisé SimCLR ;
- 2) Fine-tuning supervisé sur CIFAR-10 ;
- 3) Adaptation Test-Time Training sur CIFAR-10-C.

La Figure 1 présente une vue d’ensemble du pipeline proposé.

B. Pré-entraînement SimCLR

Nous utilisons SimCLR comme méthode d’apprentissage auto-supervisé. L’encodeur principal est un ViT-Tiny avec :

- patch size : 4×4 ;
- embedding dimension : 192 ;
- drop path : 0.1.

Deux vues augmentées sont générées pour chaque image à l’aide de transformations aléatoires :

- random crop ;
- color jitter ;
- grayscale ;
- gaussian blur.

Les représentations produites par l’encodeur sont projetées dans un espace latent de dimension 128 via une projection MLP.

C. Fine-tuning supervisé

Après le pré-entraînement SSL, le modèle est fine-tuné sur CIFAR-10.

Deux têtes sont utilisées :

- une tête de classification à 10 classes ;
- une tête auxiliaire de rotation à 4 classes.

La perte totale est définie comme :

$$\mathcal{L}_{total} = \mathcal{L}_{cls} + \lambda_{rot} \mathcal{L}_{rot}$$

où :

- $\mathcal{L}_{cls}$ est une cross-entropy classique ;
- $\mathcal{L}_{rot}$ correspond à la perte de rotation ;
- λ_{rot} contrôle l’importance de la tâche auxiliaire.

D. Test-Time Training

Pendant la phase de test, les labels ne sont pas disponibles. Cependant, la tâche auxiliaire de rotation reste exploitable.

Pour chaque image de test :

- 1) plusieurs rotations sont générées ;
- 2) la perte rotation est calculée ;
- 3) les paramètres du modèle sont mis à jour ;
- 4) la prédiction de classification est produite.

La perte rotation utilisée est :

$$\mathcal{L}_{rot} = -\sum_{c=1}^4 y_c \log(\hat{y}_c)$$

Nous étudions deux régimes d’adaptation :

- **Per-batch** : adaptation par batch complet ;
- **Per-image** : adaptation indépendante pour chaque image.

IV. EXPÉRIENCES

A. Datasets

Nous utilisons :

- CIFAR-10 pour l’entraînement supervisé ;
- CIFAR-10-C pour l’évaluation robuste.

CIFAR-10-C contient plusieurs corruptions distribuées selon cinq niveaux de sévérité.

B. Configuration expérimentale

Toutes les expériences ont été exécutées sur un GPU NVIDIA L4.

Le pré-entraînement SimCLR utilise :

- batch size : 1024 ;
- 200 époques ;
- AdamW ;
- cosine scheduler ;
- warmup sur 10 époques ;
- AMP.

TABLE I – Accuracy Top-1 sur CIFAR-10 clean

Configuration	Top-1
Baseline	0.8959
Per-batch TTT	0.8960
Per-image TTT	0.8980

V. RÉSULTATS

A. Résultats sur données propres

Le Tableau I présente les performances obtenues sur CIFAR-10 sans corruption.

Aucune dégradation n’est observée sur les données propres, ce qui valide la stabilité globale de la tête auxiliaire.

B. Résultats sous distribution shift

Le Tableau II présente les résultats moyens par niveau de sévérité.

TABLE II – Accuracy moyenne sur CIFAR-10-C

Sévérité	Baseline	Per-batch	Per-image
1	0.8673	0.8679	0.8666
2	0.8392	0.8409	0.8395
3	0.8033	0.8059	0.8033
4	0.7516	0.7575	0.7534
5	0.6803	0.6890	0.6830

Les gains du mode per-batch augmentent avec la sévérité des corruptions.

C. Effet des hyperparamètres

Les ablations montrent que :

- augmenter le nombre de steps améliore fortement le mode per-batch ;
- le mode per-image devient simultanément plus instable ;
- des learning rates trop faibles rendent l’adaptation inefficace.

VI. DISCUSSION

Nos résultats montrent que le comportement du TTT dépend fortement du régime d’adaptation utilisé.

Le mode per-batch améliore systématiquement les performances sous corruption, avec jusqu’à 67 cellules améliorées sur 70 dans la configuration la plus agressive.

À l’inverse, le mode per-image dégrade fortement les performances dans notre configuration basée sur ViT et LayerNorm.

Une hypothèse plausible est que les architectures basées sur LayerNorm ne bénéficient pas de la régularisation implicite des statistiques de batch présentes dans les architectures BatchNorm utilisées dans les travaux originaux de TTT.

Nous observons également que les gains du TTT se concentrent principalement sur les corruptions sévères telles que :

- fog ;
- gaussian noise ;
- impulse noise ;
- pixelate.

Ces corruptions préservent partiellement la structure géométrique globale de l’image, ce qui pourrait expliquer pourquoi la tâche auxiliaire de rotation reste informative.

VII. LIMITES

Notre étude présente plusieurs limitations :

- une seule seed expérimentale ;
- absence de comparaison directe avec ResNet + BatchNorm ;
- exploration partielle des hyperparamètres ;
- coût computationnel élevé du mode per-image.

VIII. CONCLUSION

Dans ce travail, nous avons étudié l’intégration du Test-Time Training dans un pipeline auto-supervisé basé sur SimCLR et ViT-Tiny.

Nos résultats montrent que le TTT per-batch améliore de manière robuste les performances sous décalage de distribution, tandis que le TTT per-image dégrade les performances dans notre configuration ViT/LayerNorm.

Ces observations suggèrent que l’efficacité du TTT dépend fortement de l’architecture de normalisation ainsi que du régime d’adaptation utilisé.

Des travaux futurs incluront :

- des comparaisons avec ResNet + BatchNorm ;
- des évaluations multi-seeds ;
- l’étude de méthodes récentes telles que Tent et ActMAD.

RÉFÉRENCES

- [1] Y. Sun et al., “Test-Time Training with Self-Supervision for Generalization under Distribution Shifts,” ICML, 2020.
- [2] Y. Liu et al., “TTT++ : When Does Self-Supervised Test-Time Training Fail or Thrive ?” NeurIPS, 2021.
- [3] T. Chen et al., “A Simple Framework for Contrastive Learning of Visual Representations,” ICML, 2020.
- [4] K. He et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” CVPR, 2020.
- [5] J.-B. Grill et al., “Bootstrap Your Own Latent,” NeurIPS, 2020.
- [6] M. Caron et al., “Emerging Properties in Self-Supervised Vision Transformers,” ICCV, 2021.
- [7] J. Han et al., “When Test-Time Adaptation Meets Self-Supervised Models,” arXiv preprint, 2025.
- [8] M. J. Mirza et al., “ActMAD : Activation Matching to Align Distributions for Test-Time-Training,” CVPR, 2023.